



Mesh Smoothing, Remeshing & Simplification

Amal Dev Parakkat
CG Group, IMAGES Team

MESH SMOOTHING

Mesh smoothing - Motivation

A technique that reduces irregularities in 3D models by adjusting vertex positions to improve the visual quality

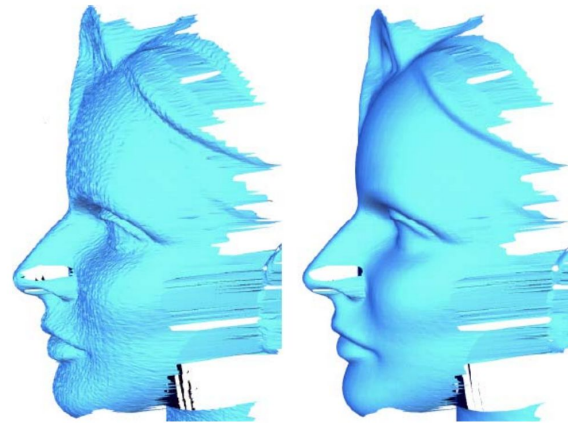
Also called as mesh filtering, denoising, or fairing

Input is a noisy mesh and output is a smooth mesh

We get noisy meshes due to the inaccuracies and imperfections in 3D scanning, data acquisition, or simulations

Can be considered as a filtering process that removes high frequency noise

"high-frequency noise" - rapid and localized changes in the geometry of the mesh



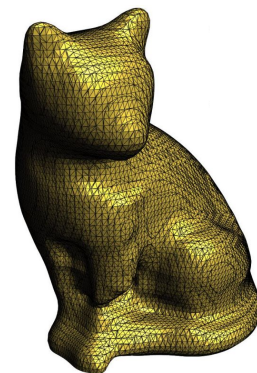
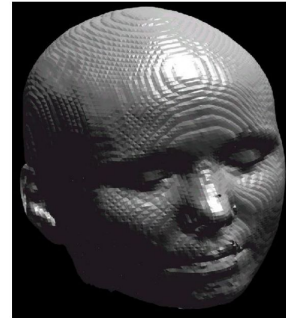
Mesh smoothing - Motivation

A mesh looks visually displeasing or “ugly” due to:

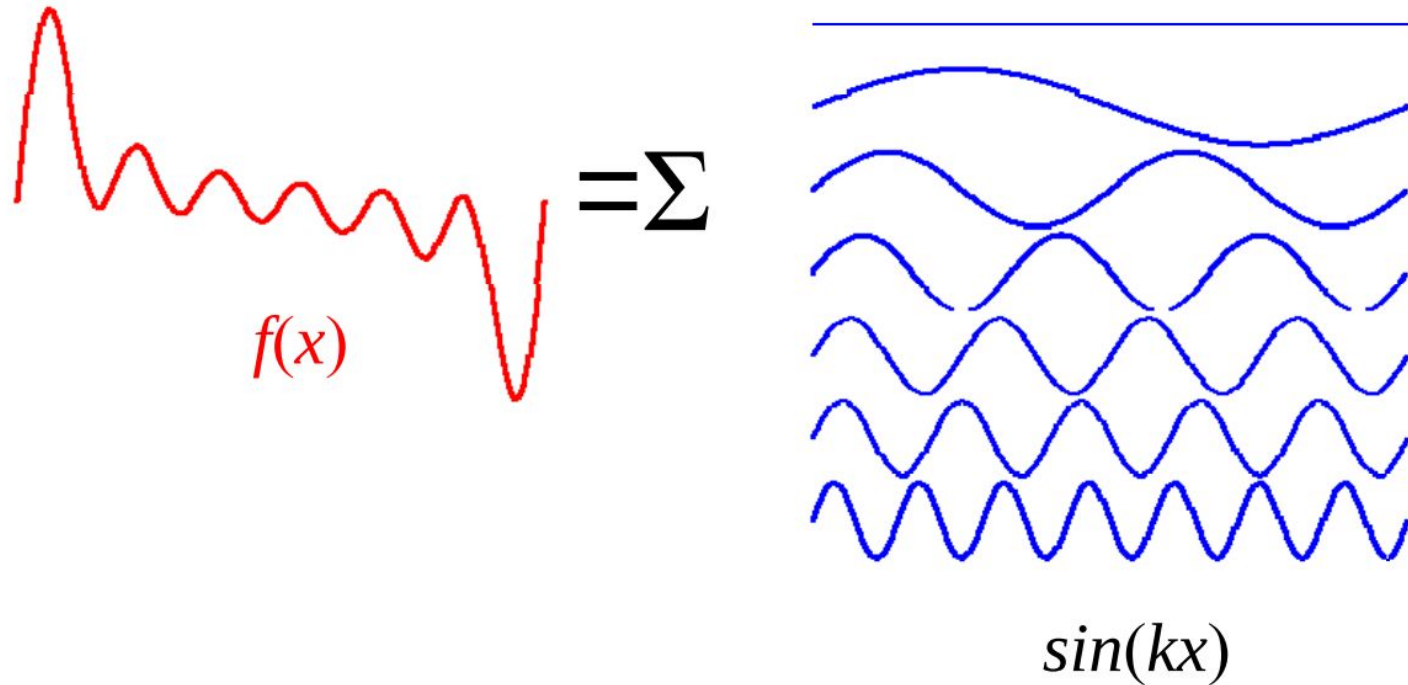
- Noisy surface - high frequency details on the surface
- Irregular mesh structure - typically due to tessellation discrepancies

Two different solutions:

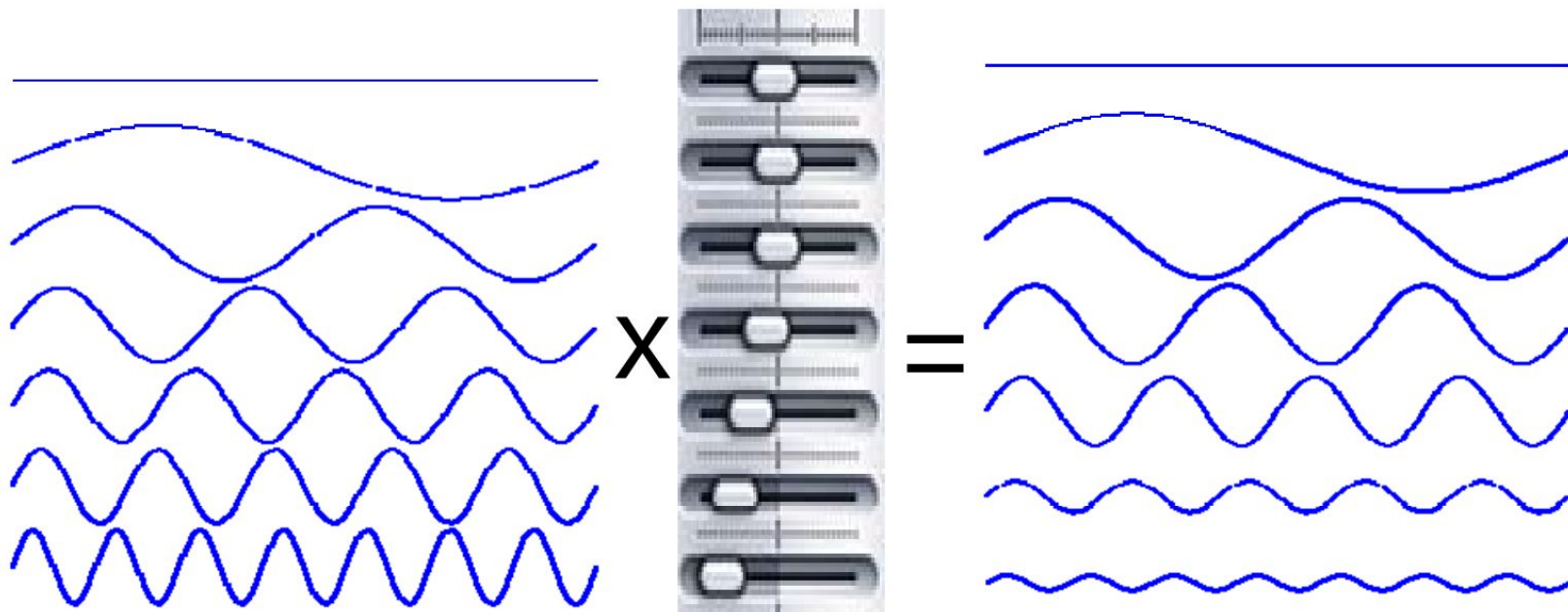
- Mesh smoothing - removing the high frequency details
- Remeshing - “Rearranging” triangles to make it look better



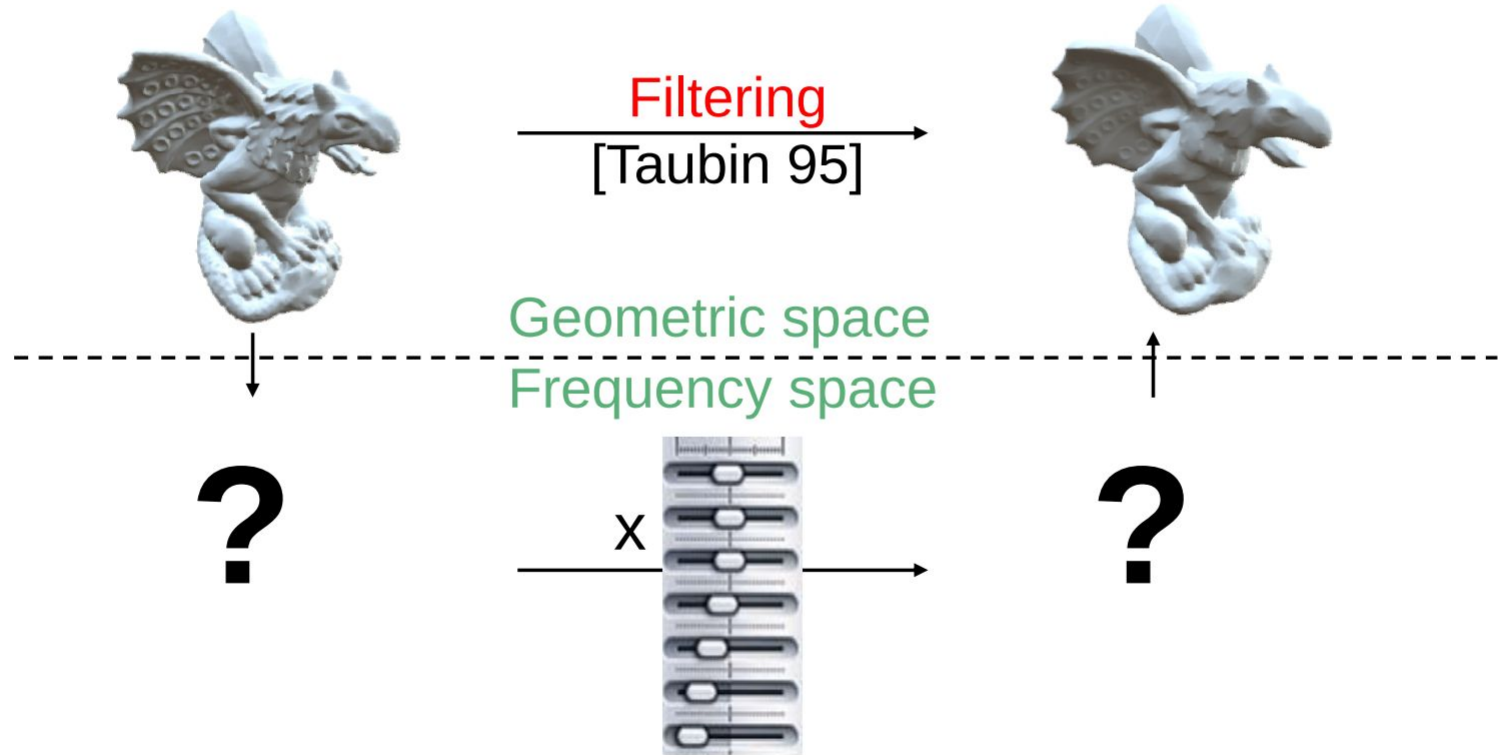
Fourier transform



Smoothing by filtering



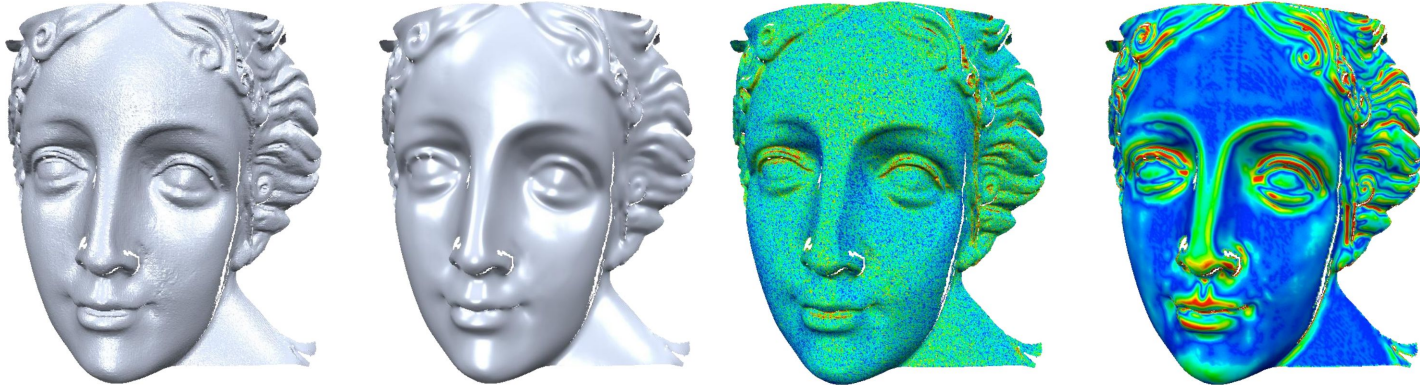
Fourier transform - how to use?



How curvature and smoothness are related?

- Principal curvature - the maximum and minimum rates of curvature in mutually perpendicular directions
- Mean curvature - the average of its principal curvatures
- Gauss curvature - the product of its principal curvatures

Noisy meshes with small irregularities introduces fluctuations in the principal curvatures, affecting the mean curvature value



*Roi Poranne

Relation b/w mean curvature and Laplacian

The mean curvature and Laplace-Beltrami are related

- Mean curvature (absolute value)

$$H = \frac{1}{2} \|\Delta_S \mathbf{x}\|$$

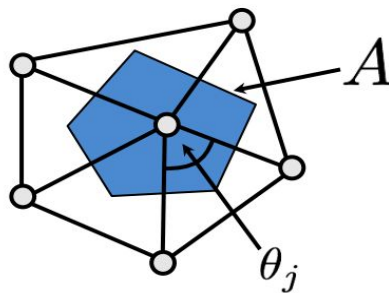
- Gaussian curvature

$$K = (2\pi - \sum_j \theta_j) / A$$

- Principal curvatures

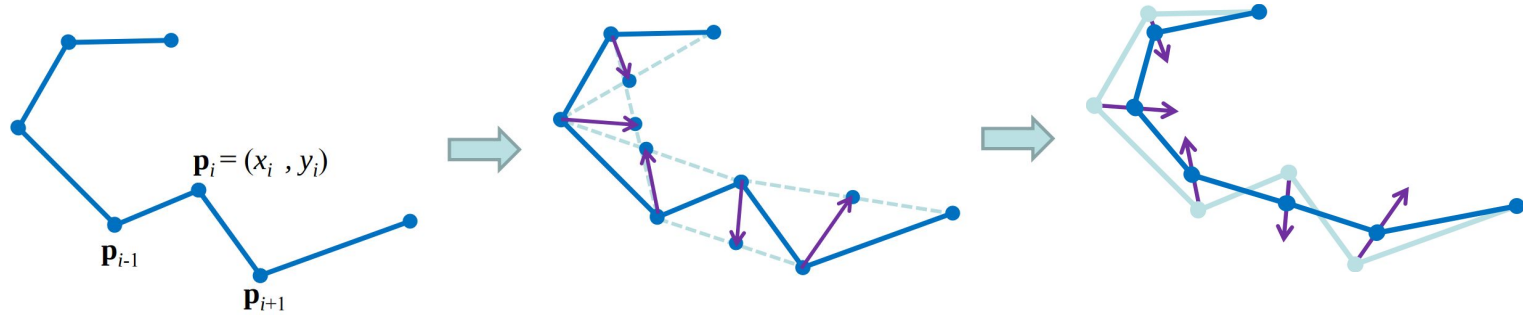
$$\kappa_1 = H + \sqrt{H^2 - K}$$

$$\kappa_2 = H - \sqrt{H^2 - K}$$



Smoothing Curves - Laplacian smoothing

Simple to understand on curves

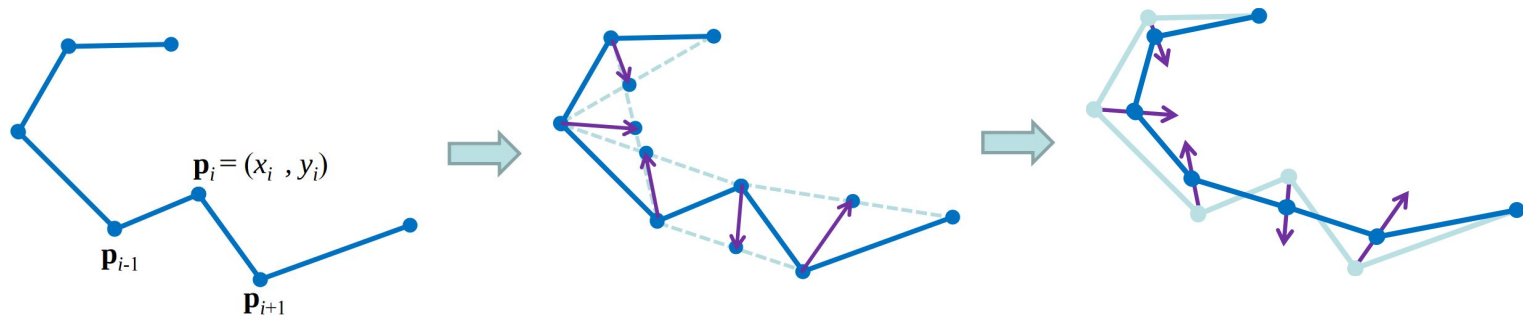


Computed as:
$$L(p_i) = \frac{1}{2}(p_{i-1} - p_i) + \frac{1}{2}(p_{i+1} - p_i)$$

Finite difference discretization of second derivative = Laplace operator in 1D

Laplacian smoothing

The idea is to iteratively update the vertex positions based on the average positions of their neighboring vertices

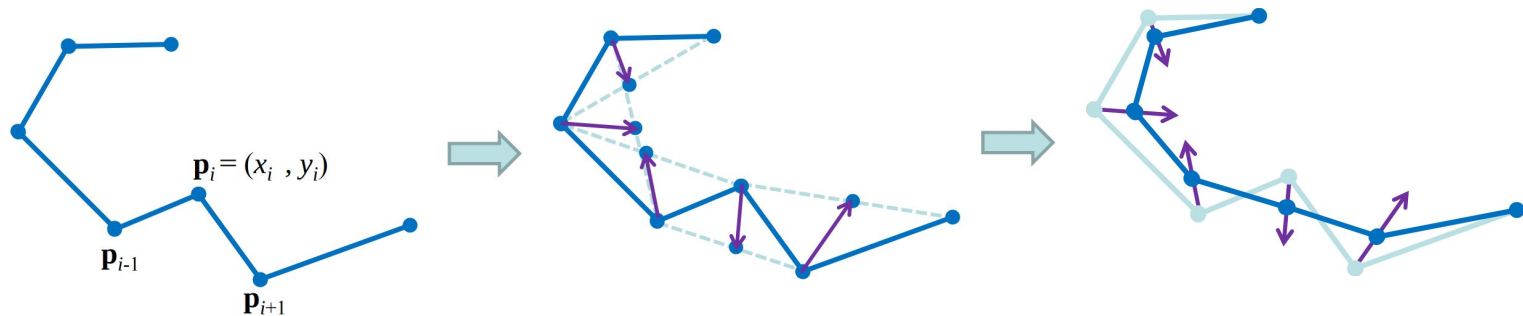


In simple terms, repeat for m iterations (on non-boundary points): $\mathbf{p}_i \leftarrow \mathbf{p}_i + \lambda L(\mathbf{p}_i)$

The value of the weight factor λ defines the resulting smoothing:

- $\lambda = 0$: No smoothing, the vertices does not move
- $\lambda = 1$: Full smoothing, the vertices move to the average position of their neighbors
- $\lambda = -1$: Inverted vertex positions, likely not desirable

Laplacian smoothing



- Smaller λ values result in milder smoothing, preserving more of the original shape
- Larger λ values increase smoothing, potentially removing finer details
- Pick smaller λ value for preserving fine details
- Value of λ is adjusted based on the visual results - typically $0 < \lambda < 1$
- Convergence of the smoothing algorithm is influenced by the choice of λ

Laplacian smoothing - Rewriting as matrices

Laplace in 1D = second derivative:

$$L(p_i) = \frac{1}{2}(p_{i-1} - p_i) + \frac{1}{2}(p_{i+1} - p_i)$$

The matrix-vector form for the whole curve:

$$Lp$$
$$p = [x, y] \in R^{n \times 2}$$

$$L = \frac{1}{2} \begin{pmatrix} -2 & 1 & & & 1 \\ 1 & -2 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & -2 & 1 \\ 1 & & & 1 & -2 \end{pmatrix}$$

Laplacian smoothing - How to extend to handle meshes?

Follow the same algorithm:

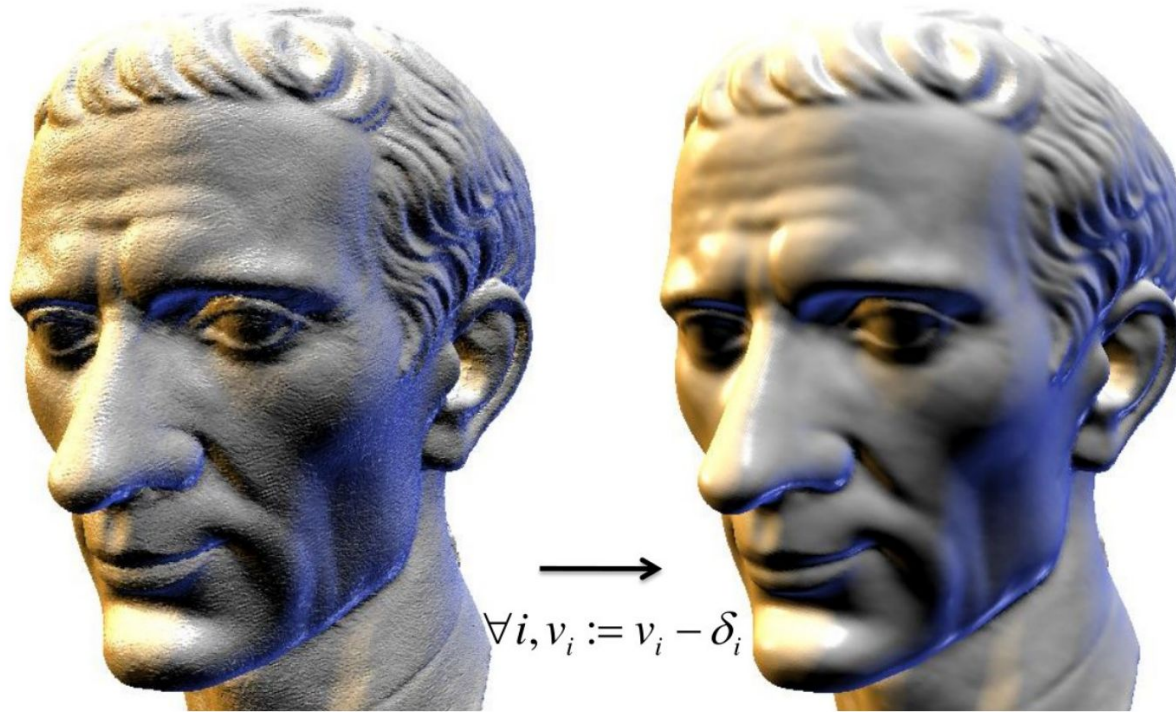
Repeat for m iterations (on non-boundary points): $\mathbf{p}_i \leftarrow \mathbf{p}_i + \lambda L(\mathbf{p}_i)$

Instead of polyline, we have triangle fans

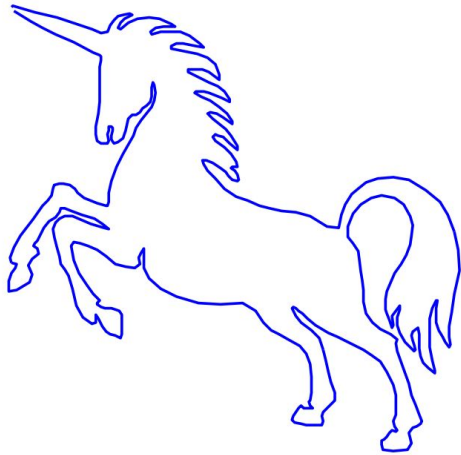
$N_i = \{k, l, m, n\}$
 $\mathbf{p}_i = (x_i, y_i, z_i)$

$$\frac{1}{2}(\mathbf{p}_{i+1} + \mathbf{p}_{i-1}) - \mathbf{p}_i$$
$$\frac{1}{|N_i|} \left(\sum_{j \in N_i} \mathbf{p}_j \right) - \mathbf{p}_i$$

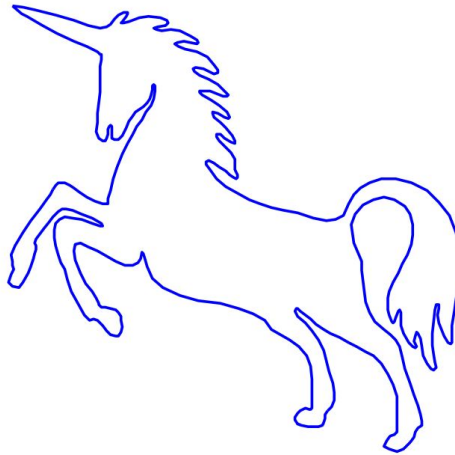
Laplacian smoothing - Extending to handle meshes



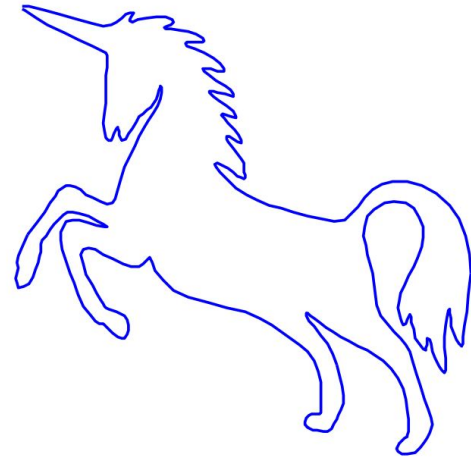
Laplacian smoothing - What happens if we keep smoothing?



Original curve

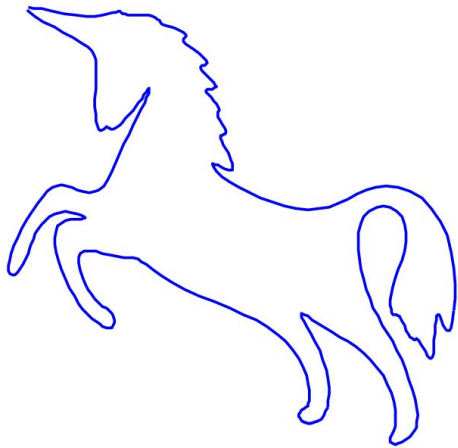


1st iteration; $\lambda=0.5$

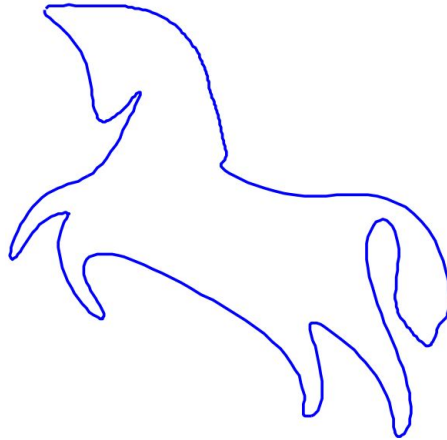


2nd iteration; $\lambda=0.5$

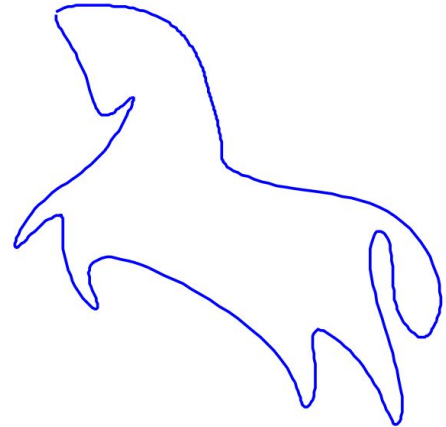
Laplacian smoothing - What happens if we keep smoothing?



8th iteration; $\lambda=0.5$

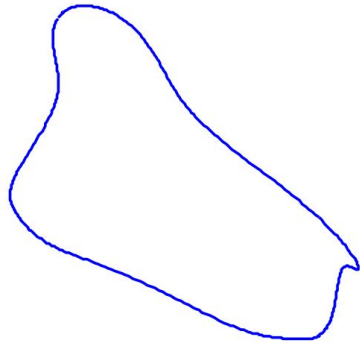


27th iteration; $\lambda=0.5$

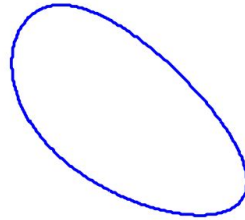


50th iteration; $\lambda=0.5$

Laplacian smoothing - What happens if we keep smoothing?



500th iteration; $\lambda=0.5$



5000th iteration; $\lambda=0.5$



50000th iteration; $\lambda=0.5$

Laplacian smoothing - Large steps in 3D



original



3 steps



6 steps



18 steps



original

Taubin smoothing

The shape goes to the opposite side when $\lambda < 1$

Simple logic: “Shrink” and then “Inflate”

Algorithmically:

Repeat for m iterations

(on non-boundary points):

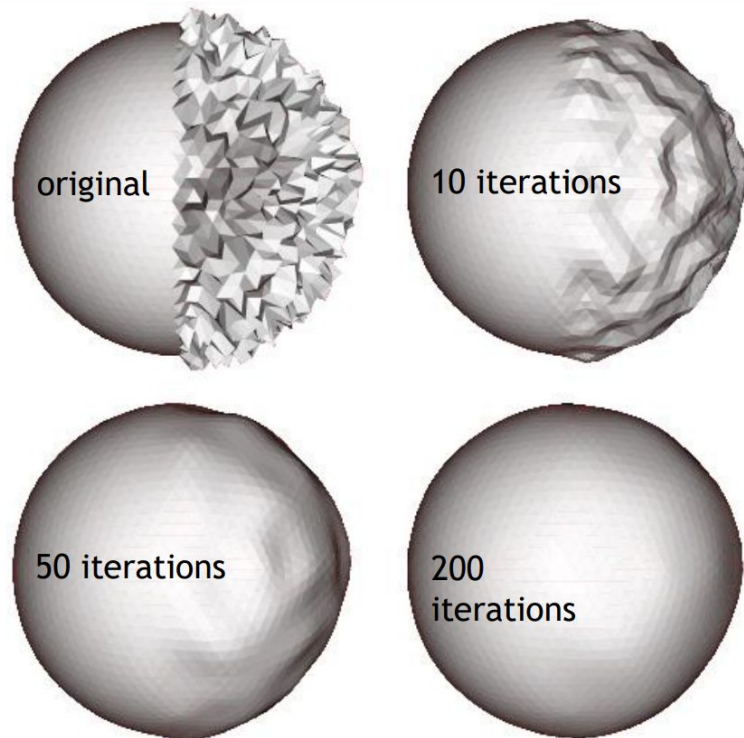
$$\mathbf{p}_i \leftarrow \mathbf{p}_i + \lambda \Delta \mathbf{p}_i$$

Shrink

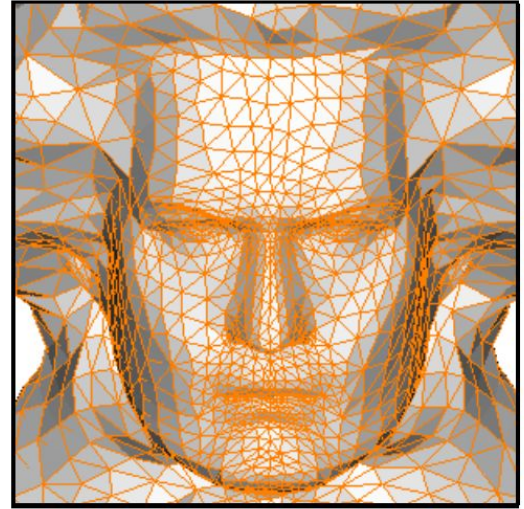
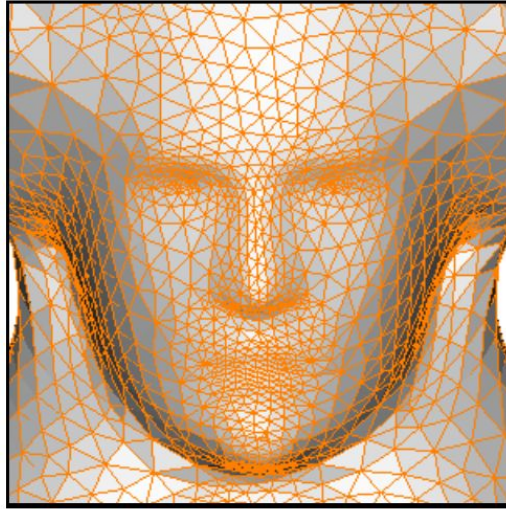
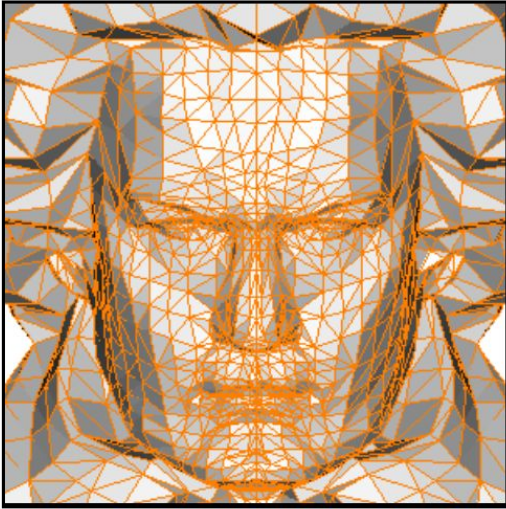
$$\mathbf{p}_i \leftarrow \mathbf{p}_i + \mu \Delta \mathbf{p}_i$$

Inflate

What should be the range of λ and μ ?



Taubin smoothing

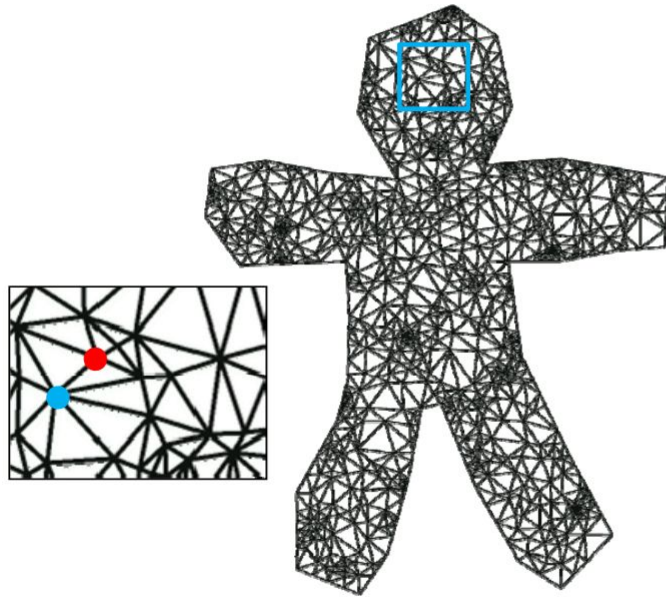


After 10 iterations of Laplacian and Taubin smoothing

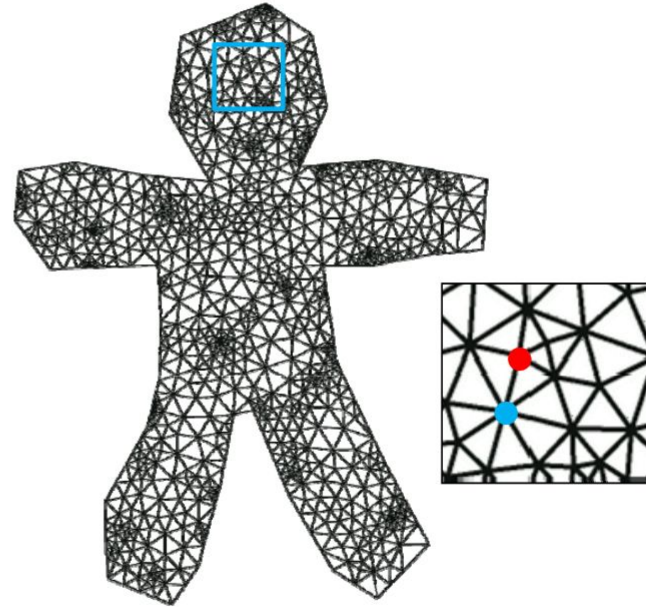
$\lambda = 0.6307$ and $\mu = -0.6732$ - manually tuned values

Laplace smoothing in 3D - Next issue

Smoothing a planar object



0 Iterations

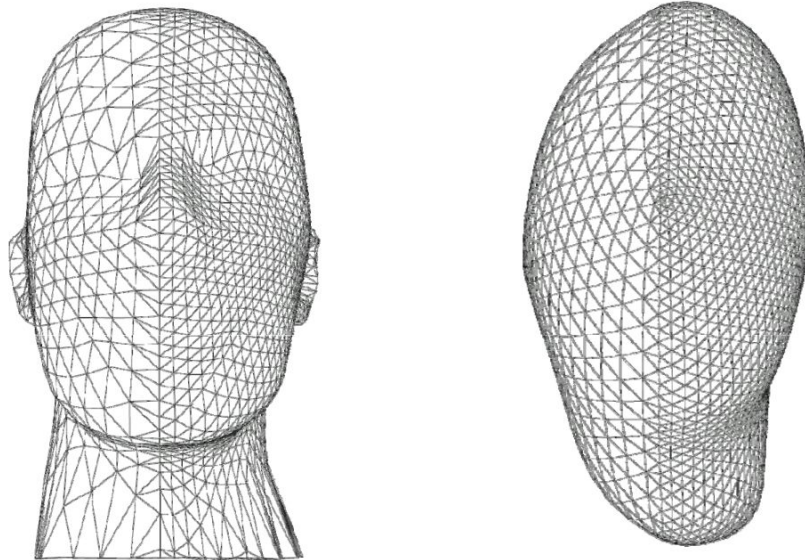


5 Iterations

Laplace smoothing in 3D - Another issue

Relation with triangle size

Asymmetric results although underlying geometry is symmetric



Possible solution - in Curves

The main issue is that all points have same effect on the new location

Back to curves

$$\frac{1}{2}(\mathbf{p}_{i+1} + \mathbf{p}_{i-1}) - \mathbf{p}_i$$



Not taking the relative distance into consideration

Same weight though one point is comparatively close

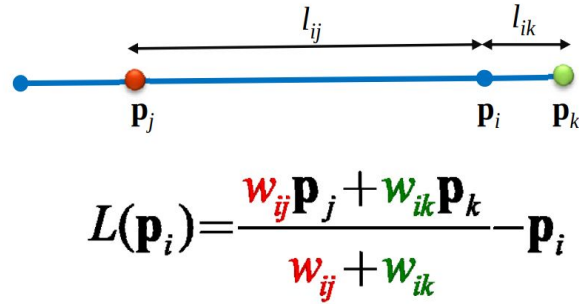
The solution is to give different weights to different points

Possible solution - in Curves

Coming up with a solution

$$w_{ij} = \frac{1}{l_{ij}}$$

$$w_{ik} = \frac{1}{l_{ik}}$$

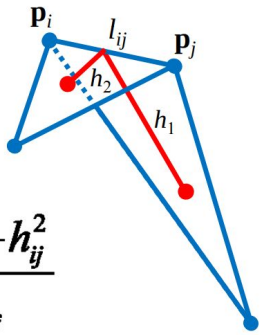
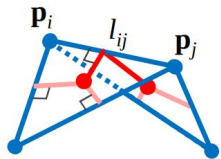


Straight curves will be invariant to smoothing

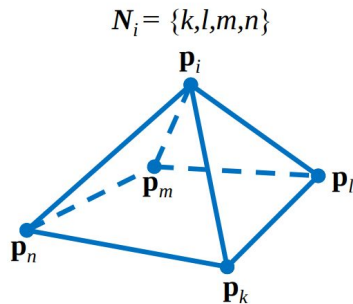
Extending solution to 3D

While extending it to 3D, we weights should reflect the angles of the triangles

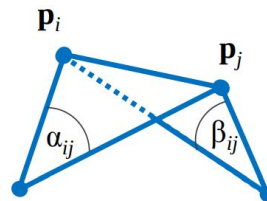
Which weights? $w_{ij} = \frac{1}{l_{ij}}$?



$$w_{ij} = \frac{h_{ij}^1 + h_{ij}^2}{l_{ij}}$$



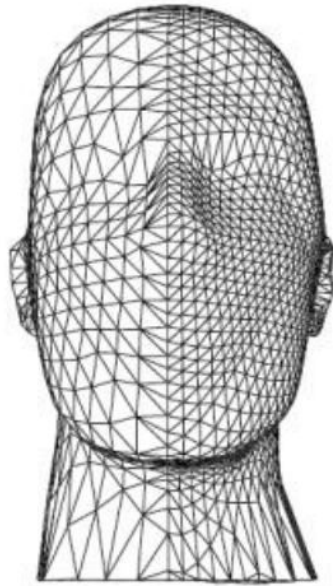
$$L(\mathbf{p}_i) = \frac{1}{\sum_{j \in N_i} w_{ij}} \left(\sum_{j \in N_i} w_{ij} \mathbf{p}_j \right) - \mathbf{p}_i$$



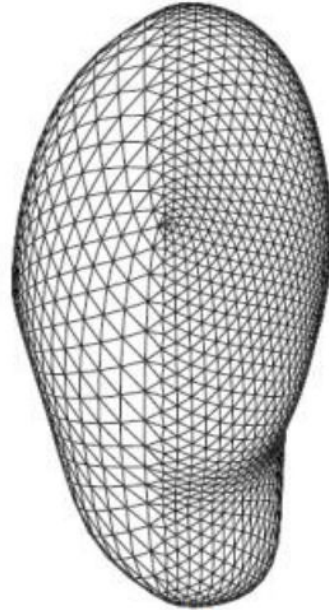
$$w_{ij} = \frac{h_{ij}^1 + h_{ij}^2}{l_{..}} = \frac{1}{2} (\cot \alpha_{ij} + \cot \beta_{ij})$$

Cotan weights

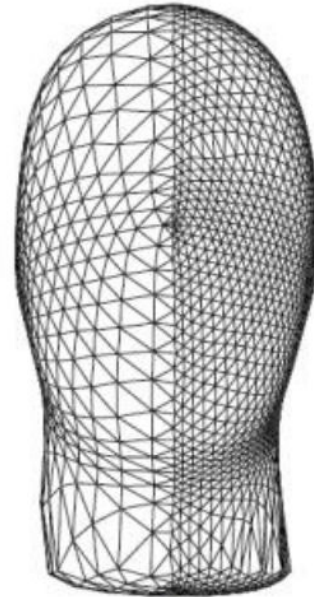
Laplace Smoothing - Uniform and Cotan weights



original



uniform

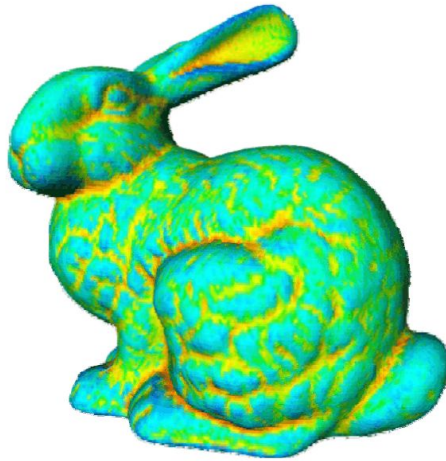


cotan

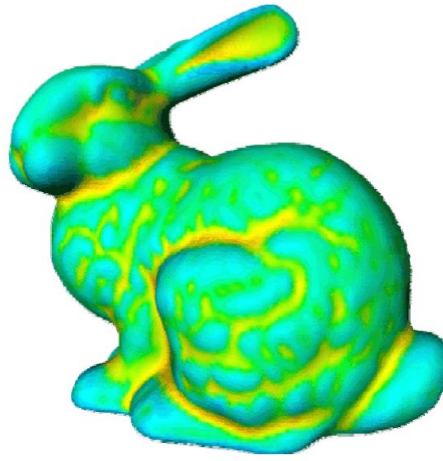
Weights are important!!!

Next problem

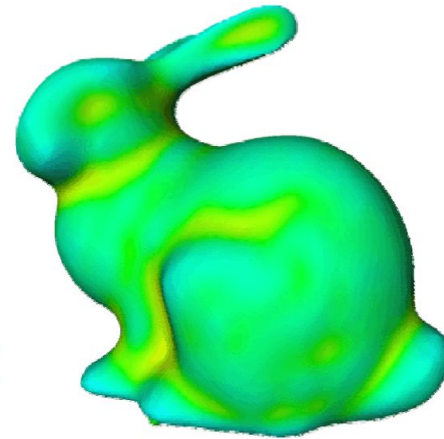
Preserving features!!!



0 Iterations



5 Iterations



20 Iterations

Introduction to Bilateral filtering

Edge-preserving smoothing technique commonly used in image processing

The main idea is to use spatial distance + signal distance

$$E(p) = \frac{1}{k(p)} \sum_{q \in \Omega} I(q) f(q - p) g(I(q) - I(p)),$$

where $k(p)$ is the normalization factor

$$k(p) = \sum_{q \in \Omega} f(q - p) g(I(q) - I(p))$$

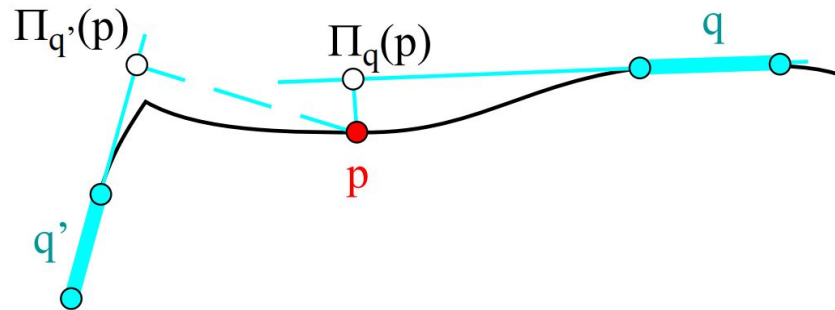


Bilateral filtering

While using it for meshes, edges become sharp features

Feature vertices are “supported” by the prediction from both sides -> Sharp features

Gaussians are used for both f and g

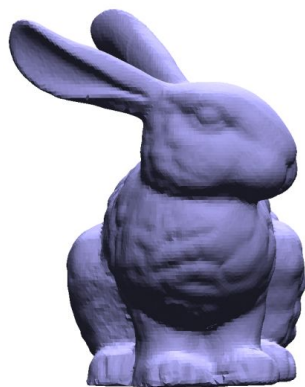
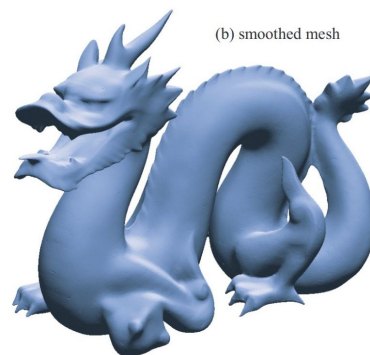
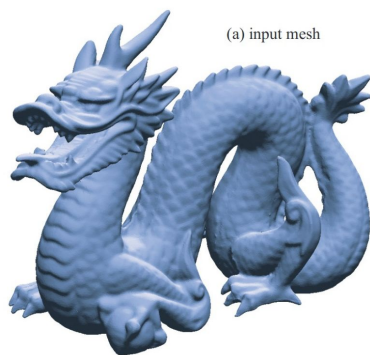


$$p' = \frac{1}{k(p)} \sum_{q \in S} \Pi_q(p) a_q f(\|c_q - p\|) g(\|\Pi_q(p) - p\|),$$

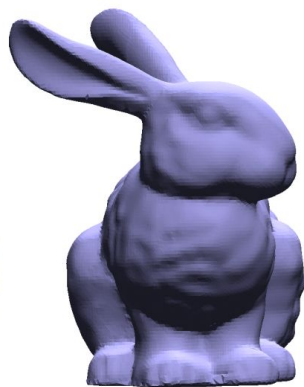
where k is a normalizing factor (sum of the weights)

$$k(p) = \sum_{q \in S} a_q f(\|c_q - p\|) g(\|\Pi_q(p) - p\|),$$

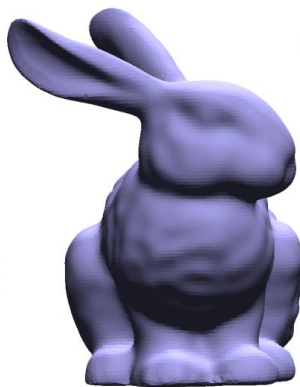
Bilateral filtering



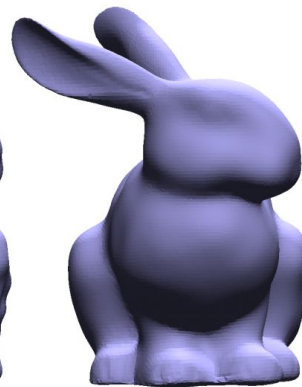
original



noise removal
narrow spatial, narrow influence
($\sigma_f = 2$, $\sigma_g = 0.2$)



smooth small features
narrow spatial, wide influence
($\sigma_f = 2$, $\sigma_g = 4$)



smooth large features
wide spatial, wide influence
($\sigma_f = 4$, $\sigma_g = 4$)

Laplacian or Bilateral filtering

Laplacian : one-ring vertex neighborhood - fast

Bilateral : large neighborhoods - slow

Laplacian : derived from « sound maths » - Mean curvature flow

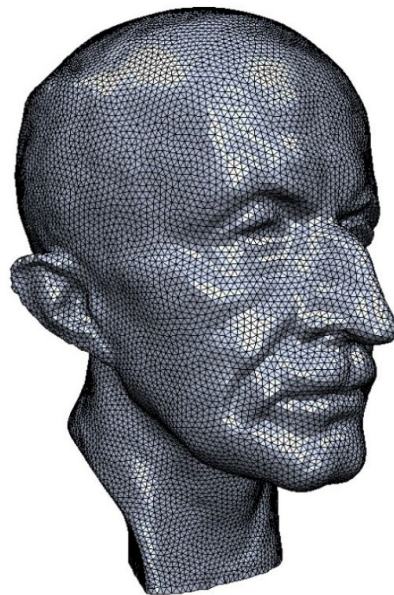
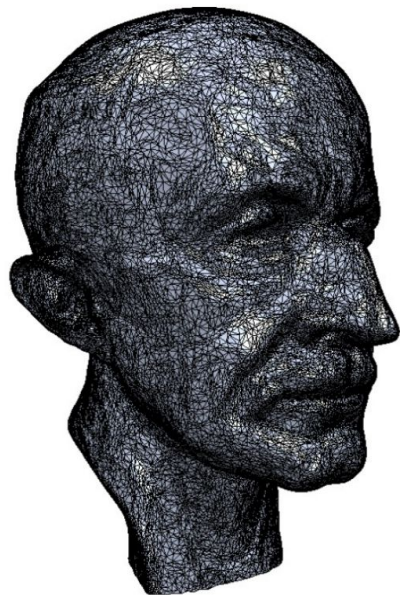
Bilateral : preservation of rich geometric frequencies - Ad-hoc inspiration from image filtering

REMESHING

Definition and motivation

Altering the mesh geometry and connectivity to improve quality

Replacing an arbitrarily structured mesh by a structured one



Remeshing - Applications

Scanned geometry - improve mesh quality

Simulation & Geometric processing - to improve size, robustness, sampling, regularity,...

Efficient shape modelling - editable models (artist-designed models)

Visualization - looking good

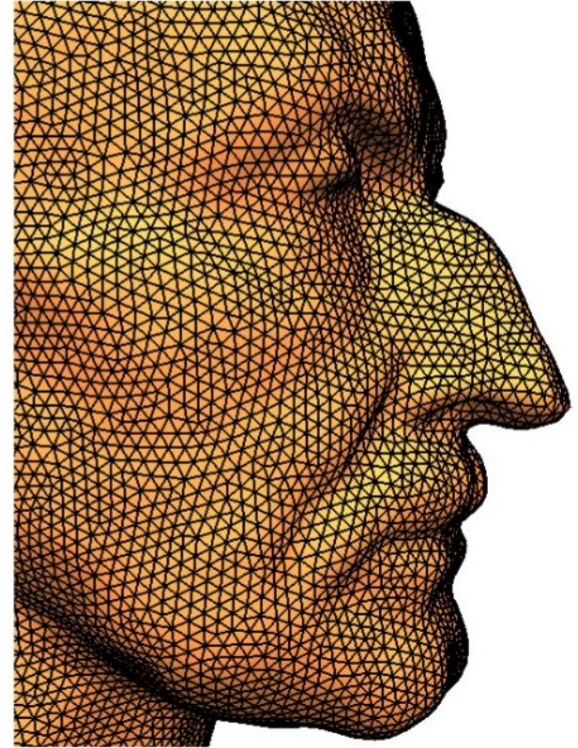
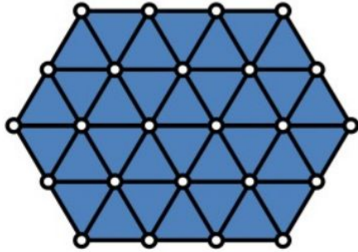
Overall, we need “good meshes”!!!

What is a good mesh?

Equal edge lengths

Equilateral triangles

Valence close to 6



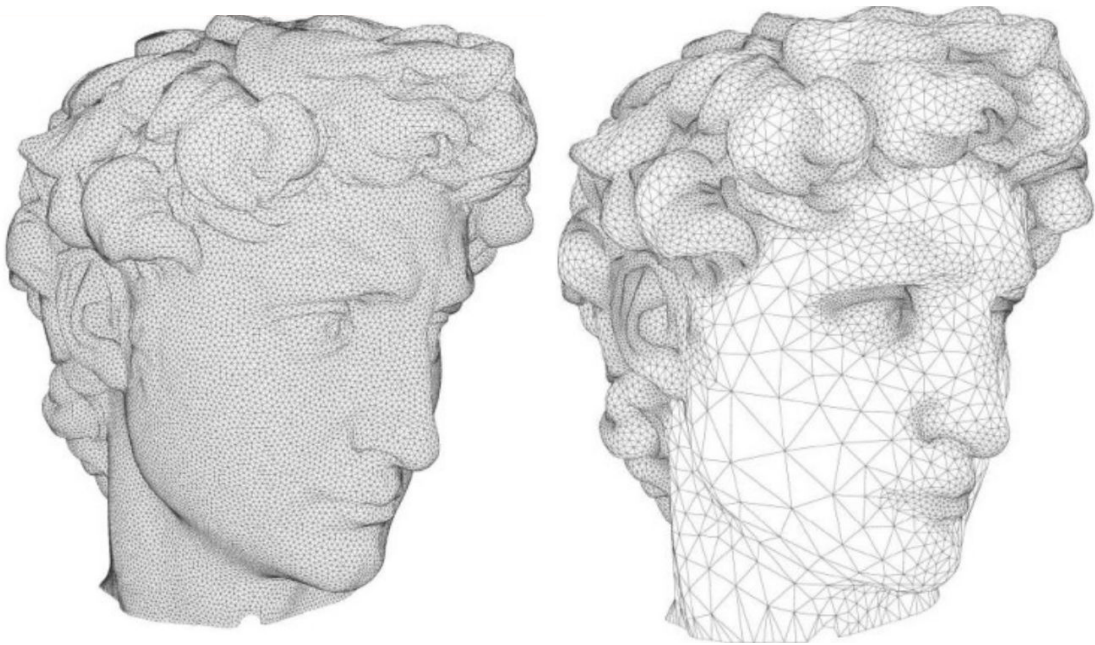
What is a good mesh?

Equal edge lengths

Equilateral triangles

Valence close to 6

Uniform vs. adaptive sampling



What is a good mesh?

Equal edge lengths

Equilateral triangles

Valence close to 6

Uniform vs. adaptive sampling

Feature preservation



What is a good mesh?

Equal edge lengths

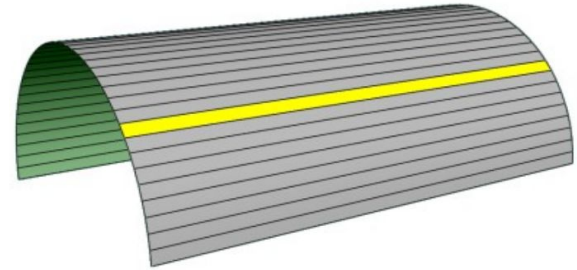
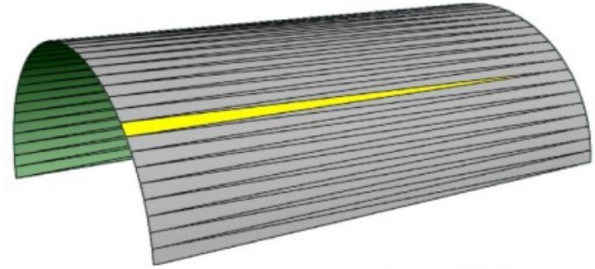
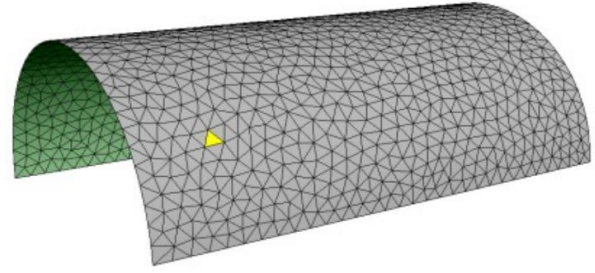
Equilateral triangles

Valence close to 6

Uniform vs. adaptive sampling

Feature preservation

Isotropic vs. anisotropic



Bounded and high aspect ratio elements

What is a good mesh?

Equal edge lengths

Equilateral triangles

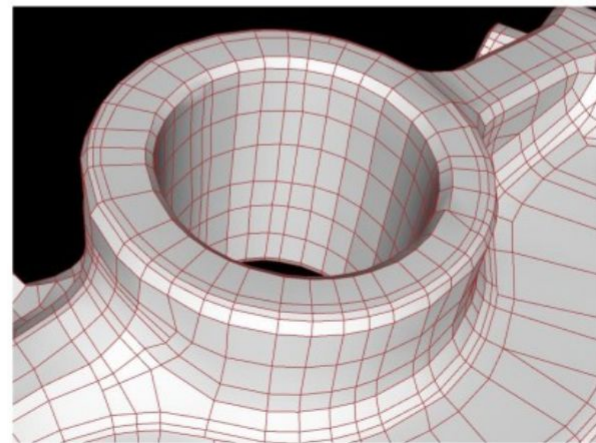
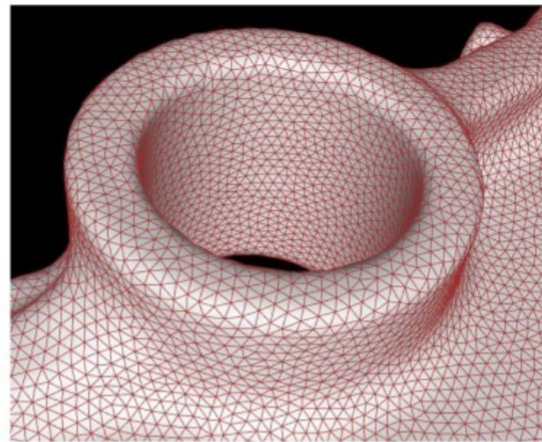
Valence close to 6

Uniform vs. adaptive sampling

Feature preservation

Isotropic vs. anisotropic

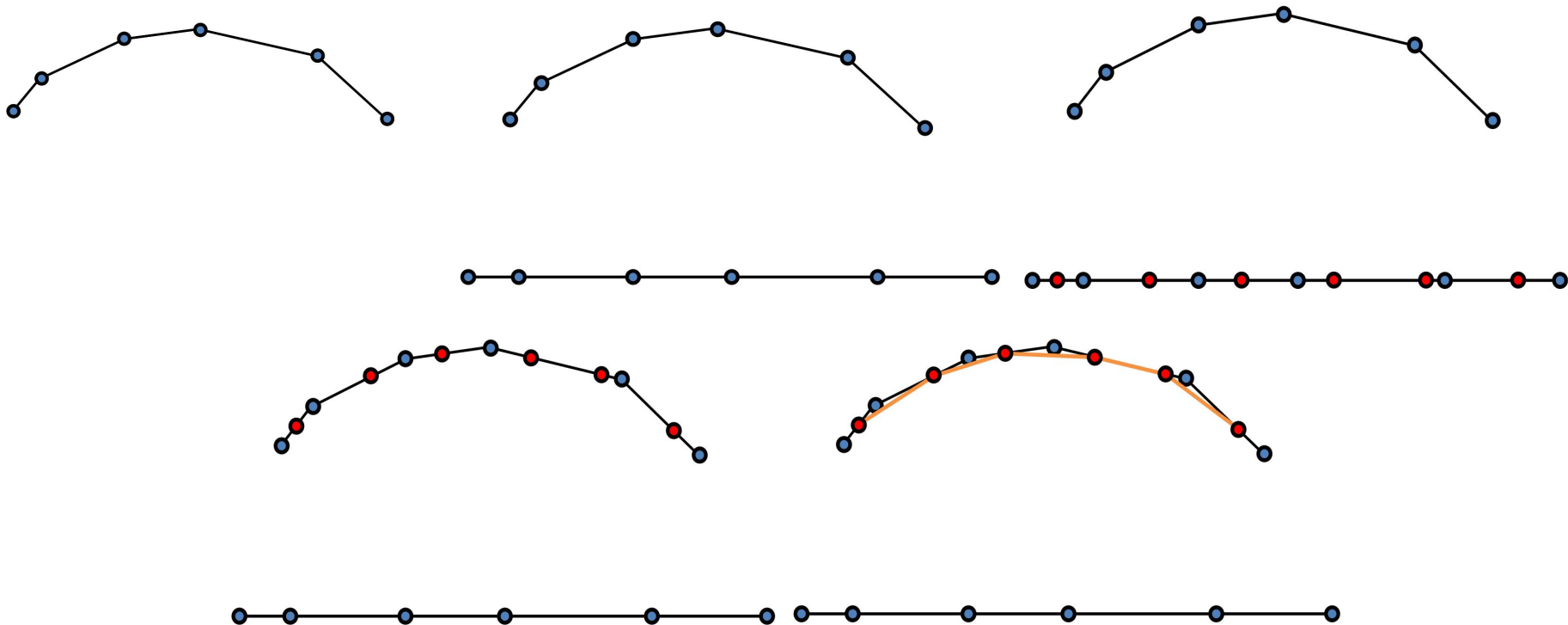
Triangles vs. quads



Parameterization-based and Surface oriented remeshing

Parameterization based Remeshing - Idea

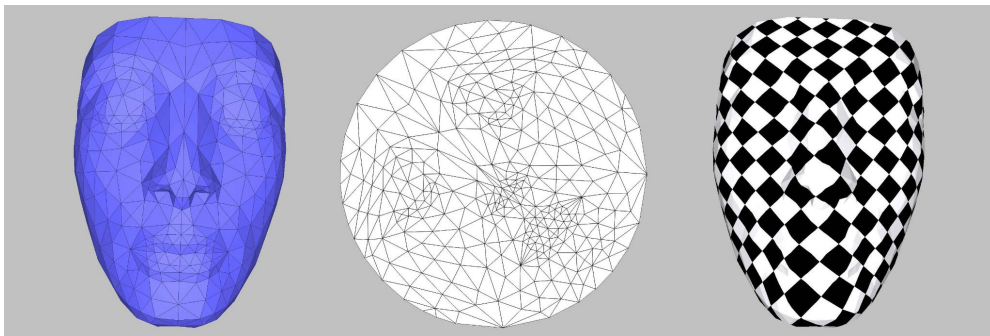
Parameterization, resampling, lifting, remeshing



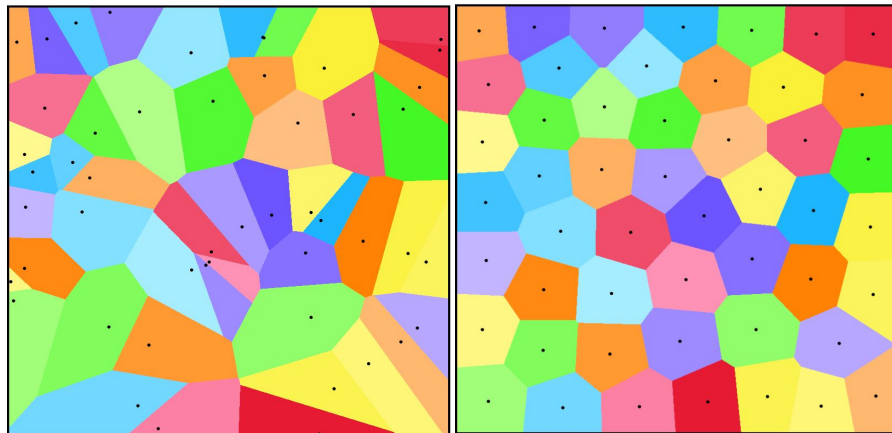
Parameterization based Remeshing

Extending it to 3D requires a way to parameterize

Introduction to Conformal Mapping and Centroidal Voronoi Diagram



*CGAL



*Roi Poranne

Mesh Parameterization: Theory and Practice
SIGGRAPH Asia 2008 Course

$$E(x_1, \dots, x_k, R_1, \dots, R_k) = \sum_{i=1}^k \int \|x - x_i\|^2$$

Parameterization based Remeshing

Conformal parameterization of input mesh

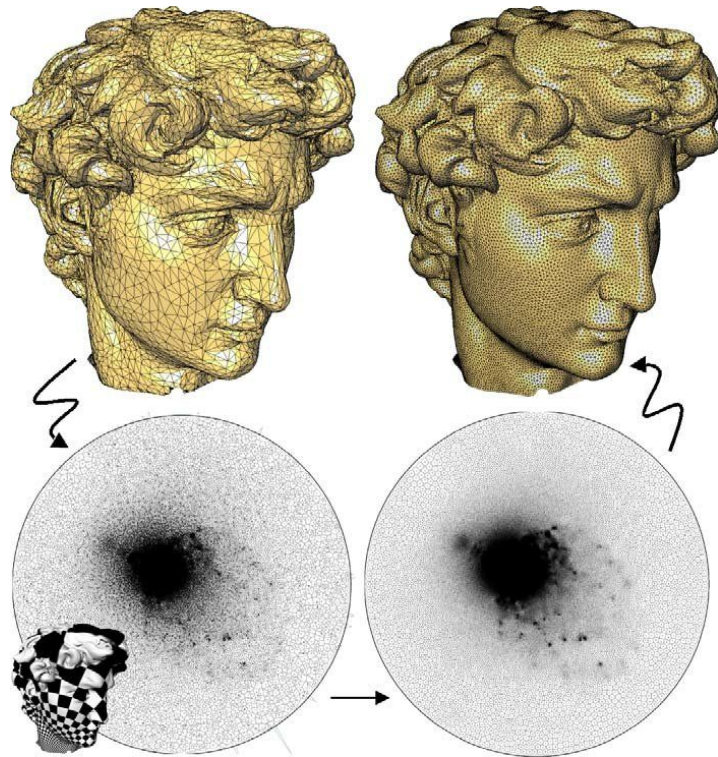
Compute local density

Perform in 2D parameter space:

- Randomly sample according to local density
- Compute CVD by Lloyd relaxation

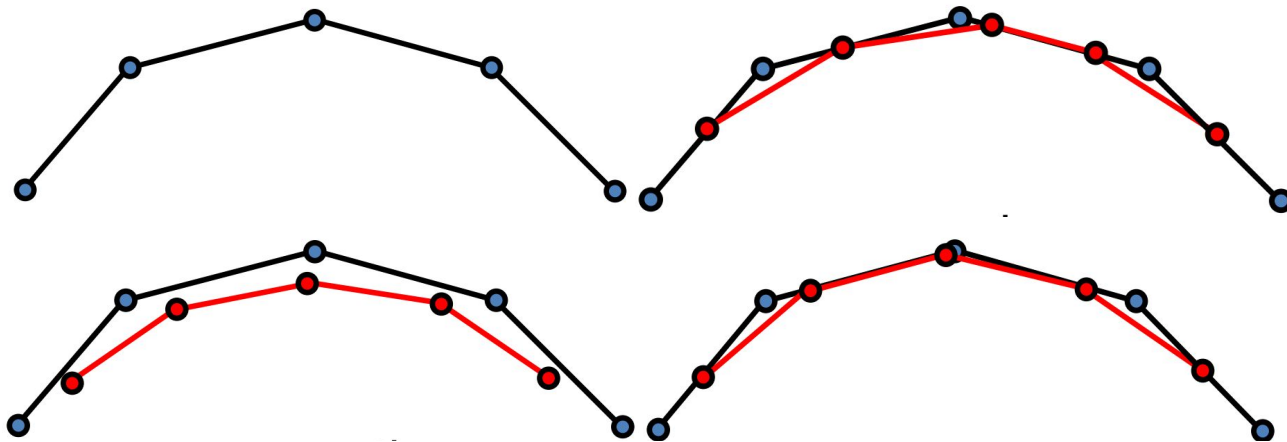
Lift 2D Delaunay triangulation to 3D

Also called as Variational remeshing



Surface oriented Remeshing

Sample, smooth, project



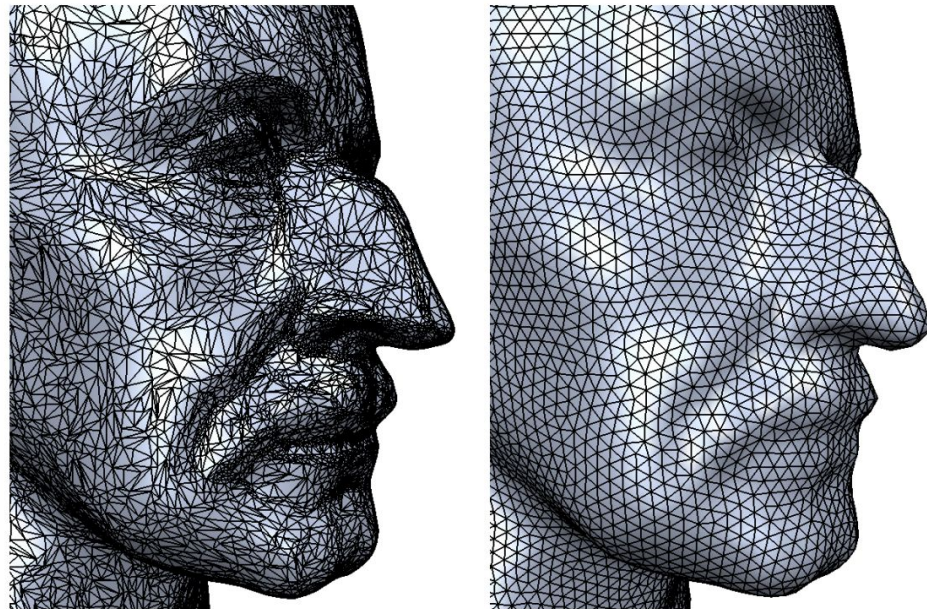
A “simple” remeshing algorithm (Botsch ‘04) - Use local operators and projections

Incremental Remeshing

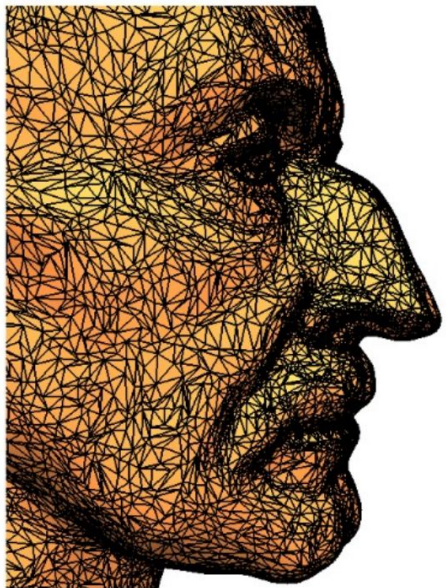
Specify target edge length L

Iterate:

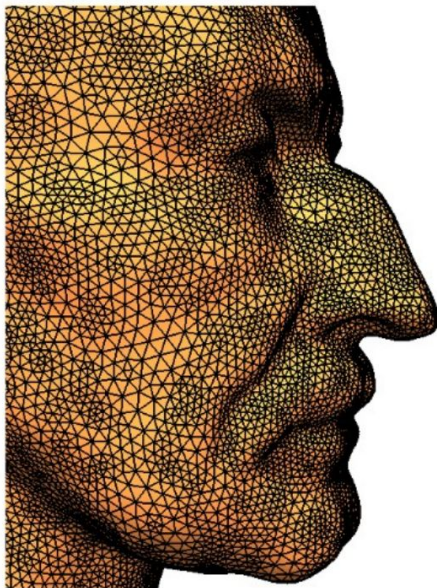
- Split edges longer than $L_{\max}$
- Collapse edges shorter than $L_{\min}$
- Flip edges to get closer to optimal valence
- Vertex shift by tangential relaxation
- Project vertices onto reference mesh



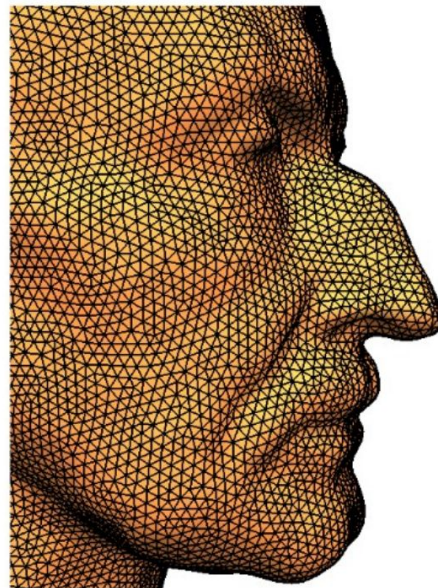
Incremental Remeshing



Original



$(\frac{1}{2}, 2)$



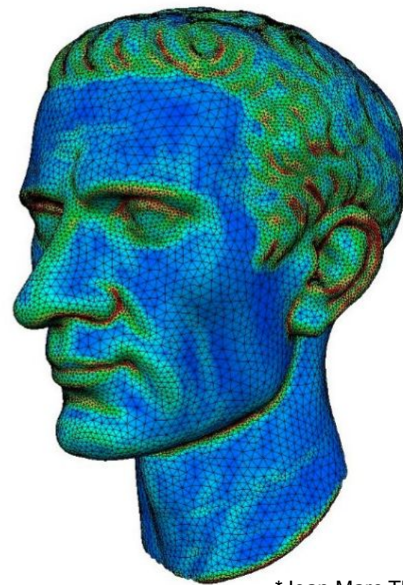
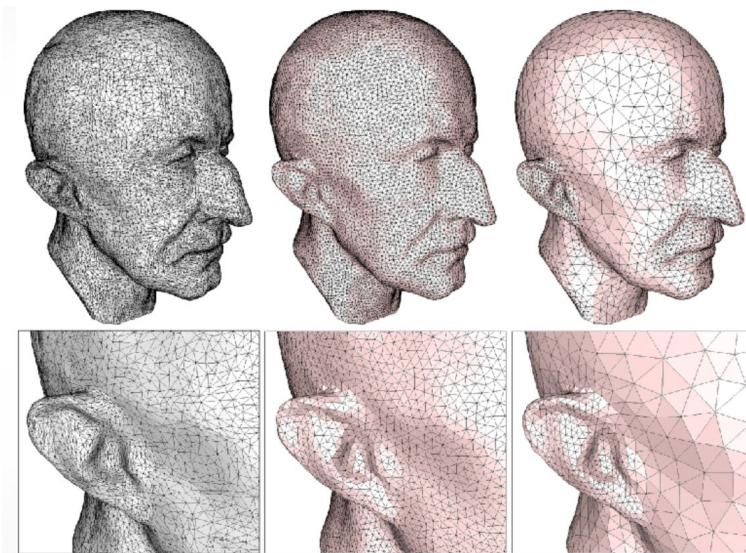
$(\frac{4}{5}, \frac{4}{3})$

Adaptive remeshing

Precompute curvatures on reference mesh

Target edge length locally determined by the curvature

Adjust/Split/Collapse criteria



*Jean Marc Thierry

*Hao Li

Feature preservation

Define feature edges / vertices

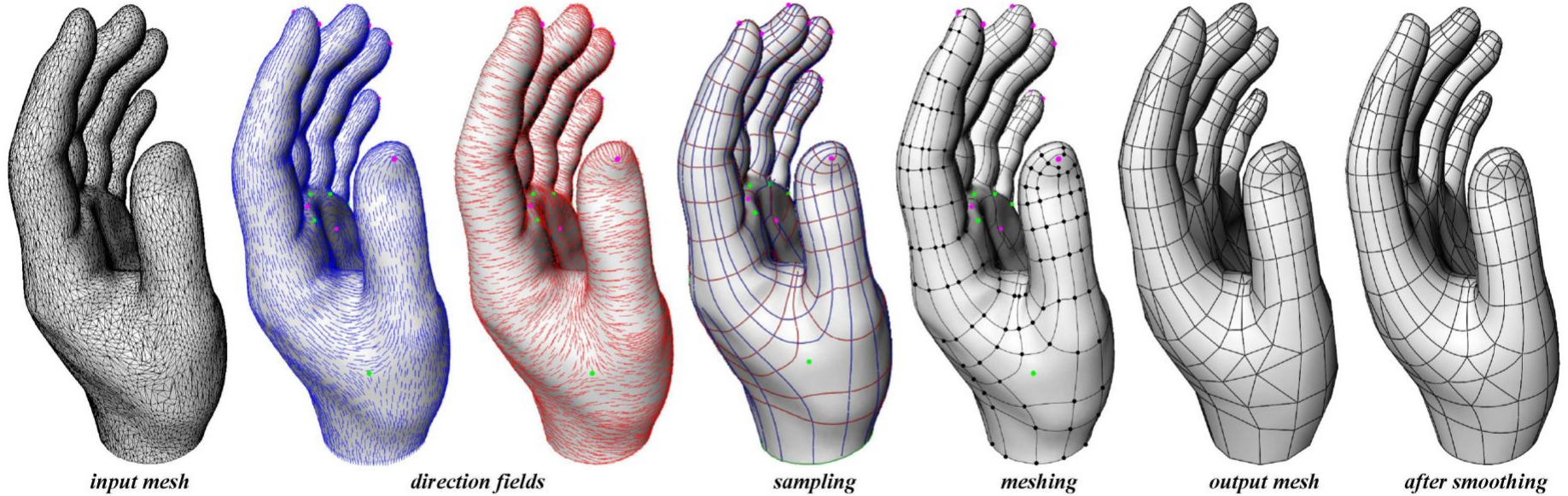
- Large dihedral angles
- Material boundaries

Adjust local operators

- Do not touch corner vertices
- Do not flip feature edges
- Collapse along features
- Univariate smoothing
- Project to feature curves



Anisotropic remeshing (Idea)



Anisotropic Polygonal Remeshing (Pierre Alliez et al.) - ACM ToG 2003

SIMPLIFICATION

Problem Statement

Given $M=(V,F)$:

Find $M'=(V',F')$: such that

1. $|V'| = n < |V|$ and $\|M-M'\|$ is minimal, or
2. $\|M-M'\| < \varepsilon$ and $|V'|$ is minimal

Very Hard!!! Look for suboptimal solution

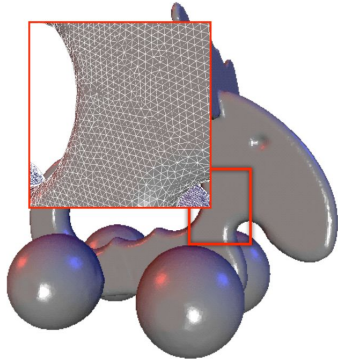
Reduce number of polygons

- faster rendering, less storage, easy to manipulate

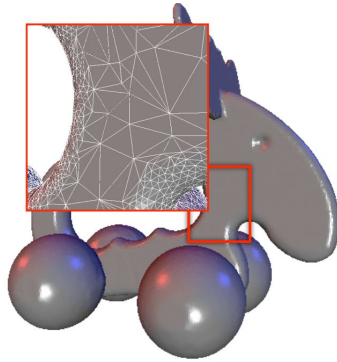
Should produce a “good” approximation



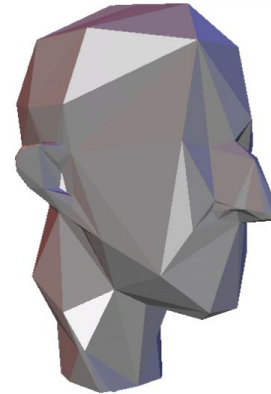
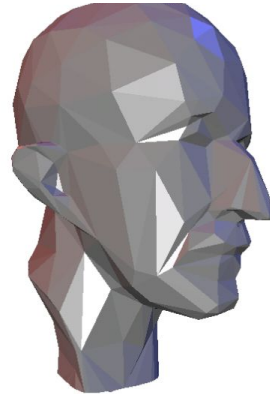
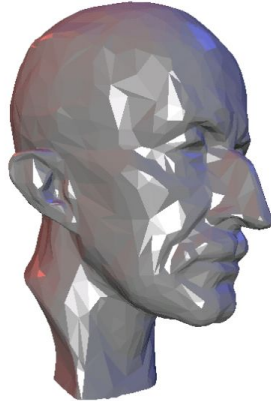
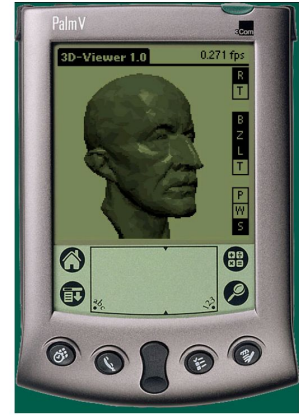
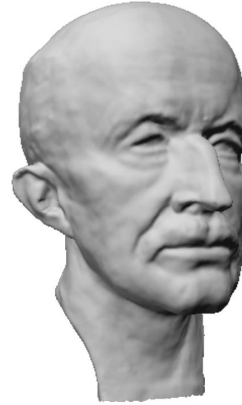
Applications



~150k triangles



~80k triangles



Two main techniques: Vertex clustering and Iterative decimation

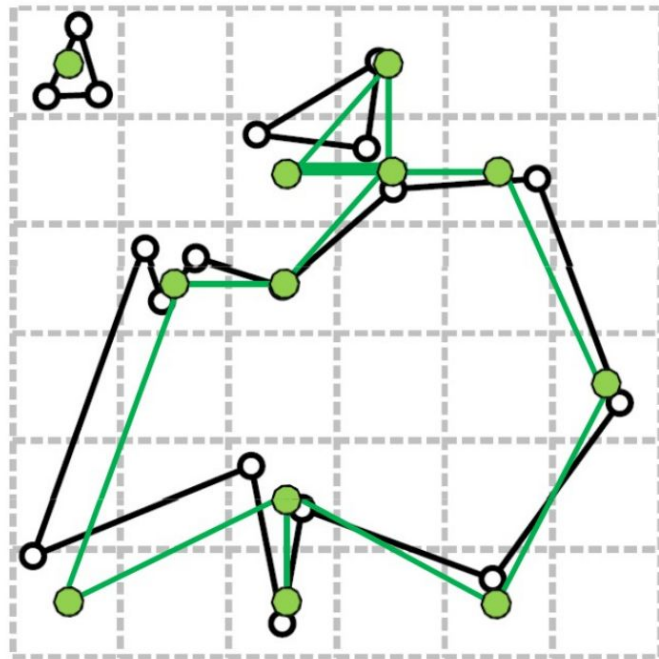
*Mark Pauly et al.

Vertex Clustering

Setup a uniform grid

Average all vertices falling into a single bin

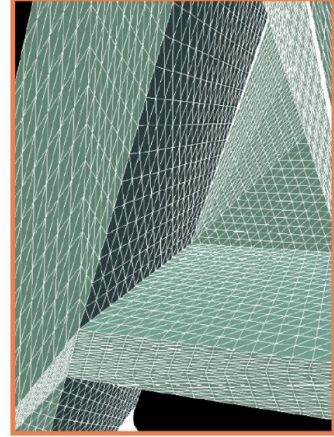
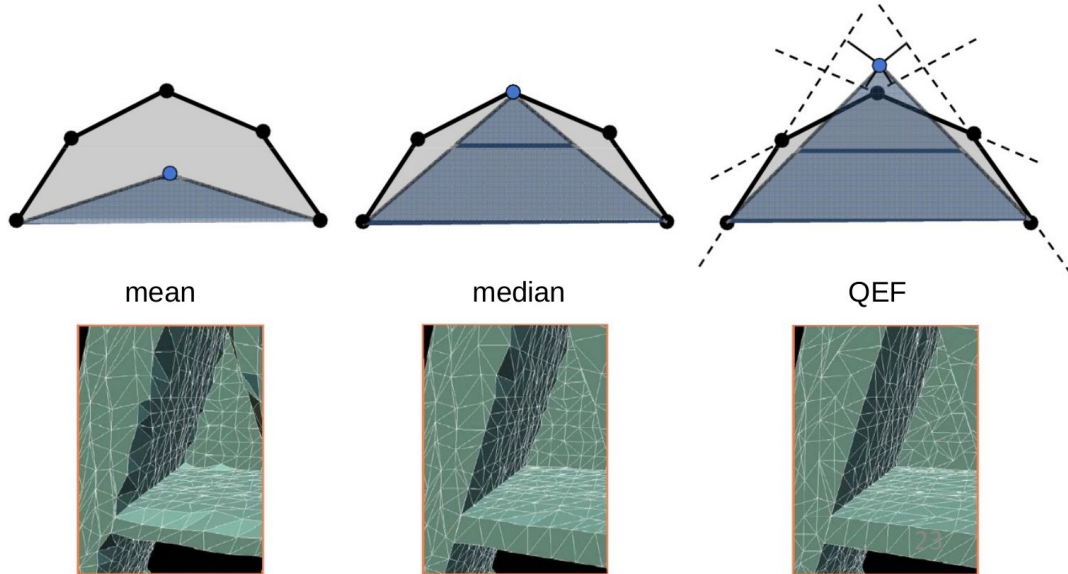
Keep non-degenerate triangles (whose corners fall into three different bins)



Vertex Clustering

The representative point inside a cell can be computed using: mean, median or QEF optimization

Quadric Error Function (QEF) optimization find a representative point that minimizes the sum of squared distances



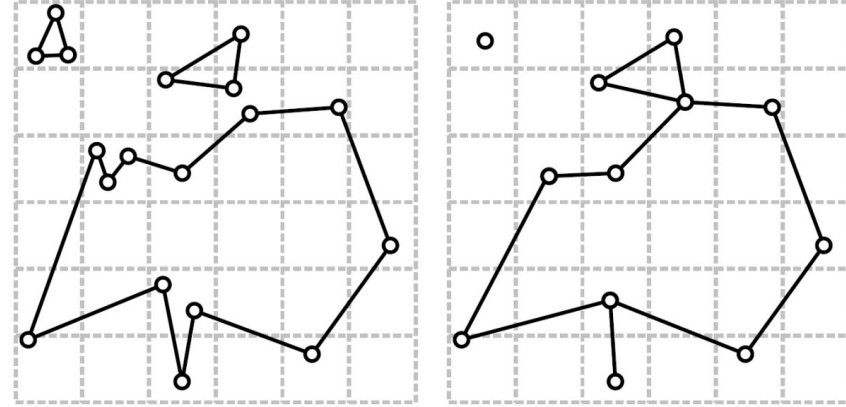
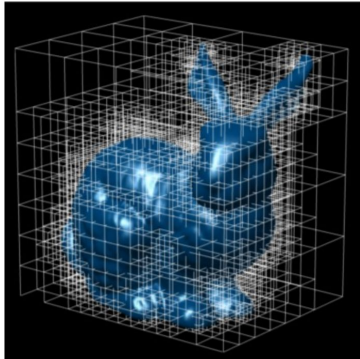
Vertex Clustering

Use 3D grids while handling meshes

Topology changes:

- If different sheets pass through one cell
- Not manifold

Grids can be replaced by adaptive structures



Iterative (Incremental) decimation - Greedy Optimization

Involves iteratively selecting and removing elements that contribute the least to the overall shape and appearance of the model

Algorithm Steps:

Repeat the following until the desired simplification is achieved:

- Calculate Error Metrics: Compute an error metric for each element
- Select Element to Remove: Identify the element with the lowest error metric
- Remove Element: Eliminate the selected element from the model and update connectivity

```
For each region
    evaluate quality after decimation
    enqueue(quality, region)
```

```
Repeat:
    pick best mesh region
    apply decimation operator
    update queue
Until no further reduction possible
```

Iterative (Incremental) decimation - Global error control

Global error control ensures that the simplification process doesn't introduce excessive errors, especially in critical regions of the model

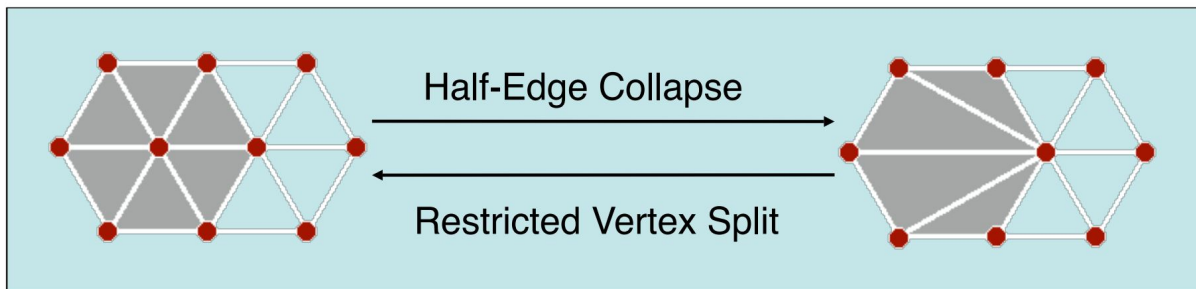
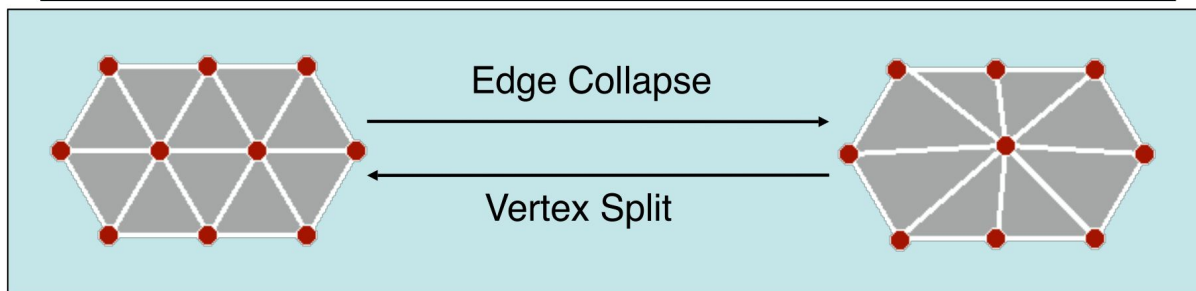
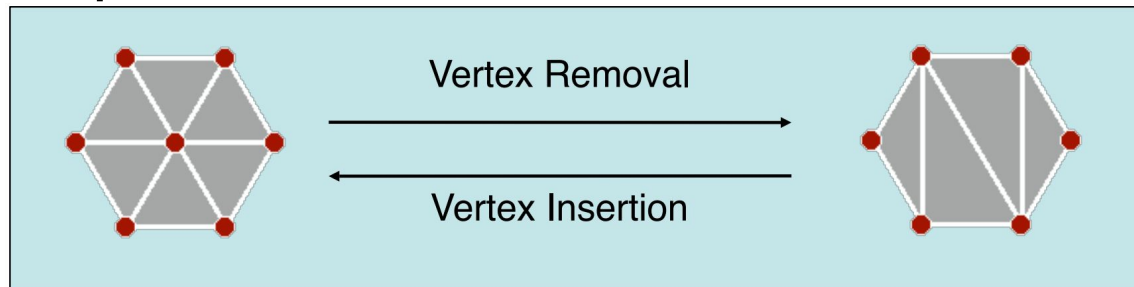
Iteratively prioritize the removal of elements with the least impact on the overall quality of the mesh

After each iteration of the simplification process, evaluate the quality metrics for the remaining mesh

```
For each region
    evaluate quality after decimation
    enqueue(quality, region)
```

```
Repeat:
    pick best mesh region
    if error <  $\epsilon$ 
        apply decimation operator
        update queue
Until no further reduction possible
```

Decimation operators



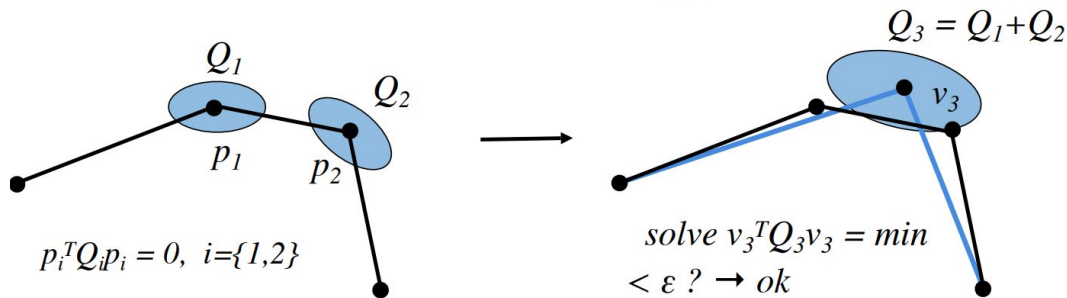
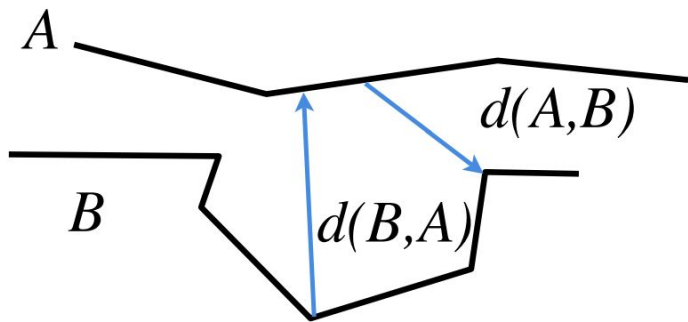
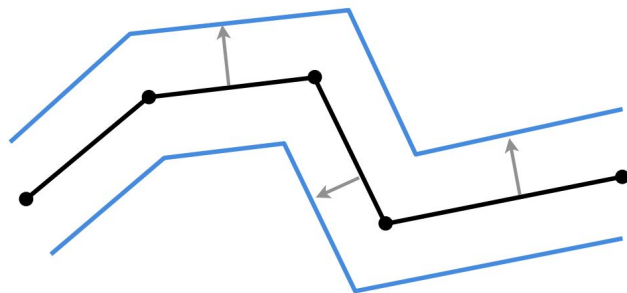
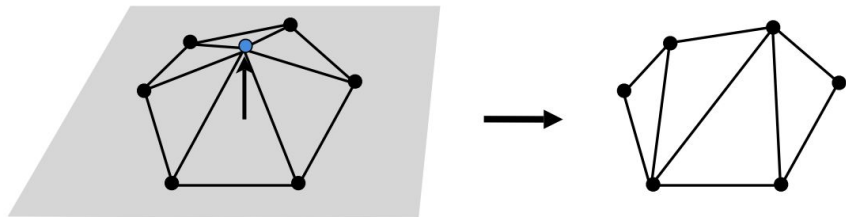
Error metrics

Local distance to mesh [Schroeder et al. 92]

Simplification envelopes [Cohen et al. 96]

Hausdorff distance (Maximum distance)

Error quadrics [Garland, Heckbert 97]



Vertex clustering vs Iterative decimation

- Vertex clustering
 - fast, but difficult to control simplified mesh
 - topology changes, non-manifold meshes
 - global error bound, but often not close to optimum
- Iterative decimation with quadric error metrics
 - good trade-off between mesh quality and speed
 - explicit control over mesh topology
 - restricting normal deviation improves mesh quality

QUESTIONS?

Credits

Inspired and most of the images from Mario Botsch et al. - Polygon Mesh Processing

IGR202 2020 course slides by Jean-Marc Thiery

Geometry Processing Algorithms course at Stanford by Mirela Ben-Chen

Shape modeling and Geometry Processing course at ETH Zurich by Roi Poranne

Many other famous academic papers :)